```
usr/bin/env python3/!#
"""
Solana Meme-Token Signal & Paper Trading Bot
.Single-file production version for free cloud hosting
"""
from __future__ import annotations

import argparse
import html
import json
import logging
import math
import os
import statistics
import sys
import threading
import time
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from http.server import HTTPServer, BaseHTTPRequestHandler
from pathlib import Path
from typing import Any, Iterable

import requests

Default File Paths #
ROOT = Path(__file__).resolve().parent
"REPORTS = ROOT / "reports
"STATE_FILE = ROOT / "state.json
"PAPER_STATE_FILE = ROOT / "paper_state.json
"PAPER_LEDGER_FILE = ROOT / "paper_ledger.jsonl

API Base Endpoints #
"DEX_BASE = "https://api.dexscreener.com
"RUG_BASE = "https://api.rugcheck.xyz
"TELEGRAM_BASE = "https://api.telegram.org
"SOLANA_NATIVE = "So11111111111111111111111111111111111111112

Default Target ID #
"DEFAULT_CHAT_ID = "8050745160

("LOG = logging.getLogger("solana-trading-bot
()SCAN_LOCK = threading.Lock
()STOP_EVENT = threading.Event

---------------------------------------------------------- #
(Health Check Server (Keeps Free Hosting Services Alive #
---------------------------------------------------------- #
```

```
:(class HealthCheckHandler(BaseHTTPRequestHandler
    :(def do_GET(self
        (self.send_response(200
("self.send_header("Content-type", "text/plain; charset=utf-8
        ()self.end_headers
(".self.wfile.write(b"Bot is active and running paper trading simulation

    :(def log_message(self, format, *args
        return  # Silence HTTP server logging

:()def start_health_server
    ((port = int(os.getenv("PORT", 8080
(server = HTTPServer(("0.0.0.0", port), HealthCheckHandler
(thread = threading.Thread(target=server.serve_forever, daemon=True
    ()thread.start
(LOG.info("Healthcheck web server started on port %d", port

# ------------------------------------------------------------
# Utility & Environment Helpers
# ------------------------------------------------------------
:def load_dotenv(path: Path = ROOT / ".env") -> None
    :()if not path.exists
        return
:()for line in path.read_text(encoding="utf-8").splitlines
    ()line = line.strip
    :if not line or line.startswith("#") or "=" not in line
        continue
    (key, value = line.split("=", 1
(("""\")os.environ.setdefault(key.strip(), value.strip().strip

:def env_float(key: str, default: float) -> float
    :try
(((return float(os.getenv(key, str(default
    :except ValueError
        return default

:def env_int(key: str, default: int) -> int
    :try
(((return int(os.getenv(key, str(default
    :except ValueError
        return default

:def get_json(url: str, *, timeout: int = 25) -> Any
response = requests.get(url, timeout=timeout, headers={"User-Agent":
    ({""solana-meme-signal-bot/1.0
    ()response.raise_for_status
        ()return response.json
```

```
def post_json(url: str, payload: dict[str, Any], *, timeout: int = 25, headers: dict[str, str] | None
                                                                                    := None) -> Any
    ({} response = requests.post(url, json=payload, timeout=timeout, headers=headers or
                                                                          ()response.raise_for_status
                                                                                ()data = response.json
                                                                  :if isinstance(data, dict) and data.get("ok") is False
                ((("raise RuntimeError(data.get("description", "remote API rejected request
                                                                  :("if isinstance(data, dict) and data.get("error
                                                                          ("error = data.get("error
        raise RuntimeError(str(error.get("message", error)) if isinstance(error, dict) else
                                                                                    ((str(error
                                                                                  return data

                                                                  :def finite_number(value: Any, default: float = 0.0) -> float
                                                                                        :try
                                                                                (number = float(value
                                                                    return number if math.isfinite(number) else default
                                                                                :(except (TypeError, ValueError
                                                                                    return default

                :def nested_number(obj: dict[str, Any] | None, key: str, default: float = 0.0) -> float
                                                                        (return finite_number((obj or {}).get(key), default

                                                                                        :def now_ms() -> int
                                                                                    (return int(time.time() * 1000

                                                                        :def age_hours(pair_created_at: Any) -> float | None
                                                                            (created = finite_number(pair_created_at, 0
                                                                                        :if created <= 0
                                                                                         return None
                                                                    (return max(0.0, (now_ms() - created) / 3_600_000

                                                                    ------------------------------------------------------------ #
                                                                                    Solana & Market Analysis Engine #
                                                                    ------------------------------------------------------------ #
                                                                        :[[def fetch_profiles() -> list[dict[str, Any
                                                                                            [] = [[candidates: list[dict[str, Any
                :("for endpoint in ("/token-profiles/latest/v1", "/token-profiles/recent-updates/v1
                                                                                                :try
                                                                                (data = get_json(DEX_BASE + endpoint
                                                                                        :(if isinstance(data, list
                                                                                    (candidates.extend(data
                                                                                        :except Exception as exc
                                                                            (LOG.warning("profile endpoint failed: %s", exc
                                                                                    {} = [[unique: dict[str, dict[str, Any
                                                                                        :for item in candidates
                                                :("if item.get("chainId") == "solana" and item.get("tokenAddress
                                                                                    unique[str(item["tokenAddress"])] = item
```

```
                                                ()return list(unique.values

:[[[def fetch_pairs(addresses: list[str]) -> dict[str, list[dict[str, Any
                                                {} = [[[result: dict[str, list[dict[str, Any
                                              :(for start in range(0, len(addresses), 30
                                                [batch = addresses[start : start + 30
                                                                    :if not batch
                                                                     continue
                                                                          :try
                                    ("{(data = get_json(f"{DEX_BASE}/tokens/v1/solana/{','.join(batch
                                                                 :(if isinstance(data, list
                                                                     :for pair in data
                                     ("" address = ((pair.get("baseToken") or {}).get("address") or
                                                                        :if address
                                                  (result.setdefault(address, []).append(pair
                                     ("" quote = ((pair.get("quoteToken") or {}).get("address") or
                                                       :if quote and quote != SOLANA_NATIVE
                                                    (result.setdefault(quote, []).append(pair
                                                                  :except Exception as exc
                                                      (LOG.warning("pair endpoint failed: %s", exc
                                                                        return result

                                :def choose_pair(pairs: Iterable[dict[str, Any]]) -> dict[str, Any] | None
                                                                  (pairs = list(pairs
                                                                        :if not pairs
                                                                      return None
                                (("return max(pairs, key=lambda p: nested_number(p.get("liquidity"), "usd

                                                                :[def rug_report(mint: str) -> dict[str, Any
                                                    ("return get_json(f"{RUG_BASE}/v1/tokens/{mint}/report

                                                :[def known_pool_owners(report: dict[str, Any]) -> set[str
                                                                    ()owners: set[str] = set
                                                  :[] for market in report.get("markets") or
                                                                      :("if market.get("pubkey
                                                    ((["owners.add(str(market["pubkey
                                                      :("for field in ("liquidityA", "liquidityB
                                                                     :(if market.get(field
                                                      (([owners.add(str(market[field
                                    :()for address, info in (report.get("knownAccounts") or {}).items
                                (("" ,"label = str((info or {}).get("type", "")) + " " + str((info or {}).get("name
                                  :(("if any(word in label.upper() for word in ("AMM", "POOL", "LOCKER
                                                                  ((owners.add(str(address
                                                                        return owners

                                                  :[def holder_stats(report: dict[str, Any]) -> tuple[float, float, int
                                                                  (pool_owners = known_pool_owners(report
                                                                        ] = percentages
```

```
                ((("finite_number(holder.get("pct
                    ([] for holder in (report.get("topHolders") or
        if str(holder.get("owner", holder.get("address", ""))) not in pool_owners
                                                                    [
                    [percentages = [x for x in percentages if x >= 0
                                    (top1 = max(percentages, default=0.0
                                ([top5 = sum(sorted(percentages, reverse=True)[:5
                                ([] return top1, top5, len(report.get("topHolders") or

                                                :[def risk_levels(report: dict[str, Any]) -> list[str
                                [([] return [str(r.get("level", "")).lower() for r in (report.get("risks") or

                                                    :def social_count(pair: dict[str, Any]) -> int
                                                    {} info = pair.get("info") or
                                    ([] return len(info.get("socials") or []) + len(info.get("websites") or

def hard_gate(pair: dict[str, Any], report: dict[str, Any], cfg: dict[str, Any]) -> tuple[bool, [[list[str
                                                                                    :[] = [reasons: list[str
                                                    (("age = age_hours(pair.get("pairCreatedAt
                                    ("liquidity = nested_number(pair.get("liquidity"), "usd
                                    ("h1_volume = nested_number(pair.get("volume"), "h1
                                                    :["if age is None or age > cfg["max_age_hours
                                                    reasons.append("العمر خارج نافذة التوكنات الجديدة")
                                                    :["if liquidity < cfg["min_liquidity
                                                    reasons.append("السيولة أقل من الحد الأدنى")
                                                    :["if h1_volume < cfg["min_h1_volume
                                                    reasons.append("حجم الساعة أقل من الحد الأدنى")
                                                    :if report.get("rugged") is True
                                    reasons.append("RugCheck وضع علامة rugged")
                :((if any(level in {"danger", "critical", "high"} for level in risk_levels(report
                                    reasons.append("RugCheck يتضمن خطراً عالياً أو حرجاً")
                                                    {} token = report.get("token") or
                                                    :("if token.get("mintAuthority
                                    reasons.append("mint authority ما زالت مفعّلة")
                                                    :("if token.get("freezeAuthority
                                    reasons.append("freeze authority ما زالت مفعّلة")
                                                    (top1, _, _ = holder_stats(report
                                                    :["if top1 > cfg["max_top_holder_pct
                                                    reasons.append("تركيز مالك كبير بعد استبعاد مجمعات السيولة")
                                                    return not reasons, reasons

def deterministic_score(pair: dict[str, Any], report: dict[str, Any], cfg: dict[str, Any]) ->
                                                                                    :[[tuple[float, dict[str, float
                                    ["age = age_hours(pair.get("pairCreatedAt")) or cfg["max_age_hours
                                    ("liquidity = nested_number(pair.get("liquidity"), "usd
                                    ("h1_volume = nested_number(pair.get("volume"), "h1
                                    ("h1_change = nested_number(pair.get("priceChange"), "h1
```

```
                {} txns = pair.get("txns") or
                {} h1_txns = txns.get("h1") or
            ("buys = nested_number(h1_txns, "buys
            ("sells = nested_number(h1_txns, "sells
                total = buys + sells
            buy_ratio = buys / total if total else 0.0
                (top1, top5, _ = holder_stats(report
                (risks = risk_levels(report
risk_penalty = min(15.0, 4.0 * sum(level in {"warn", "danger", "critical", "high"} for level in
                ((risks
                } = components
            ,(("freshness": max(0.0, 15.0 * (1.0 - age / cfg["max_age_hours"
            ,((liquidity": min(20.0, 20.0 * math.log10(max(liquidity, 1.0) / cfg["min_liquidity"] + 1.0"
volume": min(15.0, 15.0 * math.log10(max(h1_volume, 1.0) / cfg["min_h1_volume"] +"
                ,((1.0
            ,(((flow": min(10.0, max(0.0, 20.0 * (buy_ratio - 0.5"
            ,((momentum": min(15.0, max(0.0, h1_change / 10.0"
            ,((liquidity_lock": min(10.0, max(0.0, finite_number(report.get("lpLockedPct")) / 10.0"
            ,(((socials": min(5.0, float(social_count(pair"
,(distribution": max(0.0, 10.0 - max(0.0, top1 - 15.0) / 3.0 - max(0.0, top5 - 45.0) / 6.0"
            ,risk_penalty": -risk_penalty"
                {
    return max(0.0, min(100.0, sum(components.values()))), components

def normalize_candidate(profile: dict[str, Any], pair: dict[str, Any], report: dict[str, Any], cfg:
                :[dict[str, Any]) -> dict[str, Any
    (score, components = deterministic_score(pair, report, cfg
                (top1, top5, _ = holder_stats(report
                {} txns = pair.get("txns") or
                {} h1_txns = txns.get("h1") or
                } = candidate
            ,("mint": profile.get("tokenAddress"
name": (pair.get("baseToken") or {}).get("name") or (pair.get("baseToken") or"
            ,"{}).get("symbol") or "Unknown
            ,"?" symbol": (pair.get("baseToken") or {}).get("symbol") or"
                ,("pair_url": pair.get("url"
                ,("dex": pair.get("dexId"
            ,(("price_usd": finite_number(pair.get("priceUsd"
            ,("liquidity_usd": nested_number(pair.get("liquidity"), "usd"
            ,("volume_h1_usd": nested_number(pair.get("volume"), "h1"
            ,("volume_h24_usd": nested_number(pair.get("volume"), "h24"
            ,("change_m5_pct": nested_number(pair.get("priceChange"), "m5"
            ,("change_h1_pct": nested_number(pair.get("priceChange"), "h1"
            ,("change_h24_pct": nested_number(pair.get("priceChange"), "h24"
            ,(("buys_h1": int(nested_number(h1_txns, "buys"
            ,(("sells_h1": int(nested_number(h1_txns, "sells"
                ,(("fdv_usd": finite_number(pair.get("fdv"
            ,(("market_cap_usd": finite_number(pair.get("marketCap"
```

```
                ,(("age_hours": age_hours(pair.get("pairCreatedAt"
                                ,top_holder_pct_ex_pool": top1"
                                ,top5_holders_pct_ex_pool": top5"
                                ,("reported_holder_count": report.get("totalHolders"
                                lp_locked_pct": finite_number(report.get("lpLockedPct"),"
max((finite_number(((m.get("lp") or {}).get("lpLockedPct"))) for m in (report.get("markets") or
                                ,(([])), default=0.0
                ,(("rugcheck_score_normalised": finite_number(report.get("score_normalised"
                                ,[] rugcheck_risks": report.get("risks") or"
                                ,(risk_levels": risk_levels(report"
                ,("mint_authority": (report.get("token") or {}).get("mintAuthority"
                ,("freeze_authority": (report.get("token") or {}).get("freezeAuthority"
                ,"" description": profile.get("description") or"
                ,[] socials": (pair.get("info") or {}).get("socials") or"
                ,[] websites": (pair.get("info") or {}).get("websites") or"
                ,(raw_score": round(score, 2"
                ,{()score_components": {k: round(v, 2) for k, v in components.items"
                                                {
                        (allowed, reasons = hard_gate(pair, report, cfg
                        candidate["hard_gate_passed"] = allowed
                        candidate["gate_reasons"] = reasons
                        return candidate

:[def ai_assess(candidate: dict[str, Any], cfg: dict[str, Any]) -> dict[str, Any
                                                } = fallback
,"action": "SIGNAL" if candidate["raw_score"] >= cfg["signal_threshold"] else "WATCH"
                ,((("confidence": int(max(0, min(100, candidate["raw_score"
                "thesis": "اجتاز المرشح بوابة الفحص، ويتم اختبار حركته عبر محاكي التداول الورقي.",
                "risks": ["تقلب عالي جداً", "إمكانية تغيير السيولة", *candidate.get("gate_reasons", [])],
                "invalidation": "إلغاء المراقبة في حال انخفاض السيولة أو ظهور مخاطر جديدة.",
                "entry_plan": "دخول ورقي افتراضي بناءً على المحاكاة الحالية.",
                "exit_plan": "إغلاق افتراضي عند جني الأرباح (TP1/TP2) أو وقف الخسارة.",
                "rationale": "تقييم خوارزمي تلقائي للمحاكاة.",
                                                {
        if not cfg["ai_enabled"] or not os.getenv("OPENAI_API_KEY") or not
                                :("os.getenv("OPENAI_API_BASE
                                        return fallback
                                                :try
                                                } = schema
                                        ,"type": "object"
                                        } :"properties"
                ,{["action": {"type": "string", "enum": ["SIGNAL", "WATCH", "AVOID"
                ,{confidence": {"type": "integer", "minimum": 0, "maximum": 100"
                                ,{"thesis": {"type": "string"
                ,{{"risks": {"type": "array", "items": {"type": "string"
                                ,{"invalidation": {"type": "string"
                                ,{"entry_plan": {"type": "string"
                                ,{"exit_plan": {"type": "string"
```

```
                ,{"rationale": {"type": "string"
                                                ,{
required": ["action", "confidence", "thesis", "risks", "invalidation", "entry_plan","
                                                ,[""exit_plan", "rationale
                                                ,additionalProperties": False"
                                                {
                system = "أنت محلل مخاطر لتوكنات سولانا. اخرج JSON حصراً باللغة العربية."
                                                } = payload
                ,("model": os.getenv("AI_MODEL", "gpt-4o-mini"
messages": [{"role": "system", "content": system}, {"role": "user", "content":"
                                                ,[{(json.dumps(candidate, ensure_ascii=False
response_format": {"type": "json_schema", "json_schema": {"name": "signal", "strict":"
                                                ,{{True, "schema": schema
                                                {
    result = post_json(os.getenv("OPENAI_API_BASE").rstrip("/") + "/chat/completions",
({"{('payload, timeout=45, headers={"Authorization": f"Bearer {os.getenv('OPENAI_API_KEY
                ("return json.loads(result["choices"][0]["message"]["content
                                        :except Exception as exc
            (LOG.warning("AI query failed, using deterministic fallback: %s", exc
                                        return fallback

                                        ---------------------------------------------------------- #
                                        Formatting Helpers #
                                        ---------------------------------------------------------- #
                :[def signal_levels(candidate: dict[str, Any]) -> dict[str, float | None
                                price = candidate.get("price_usd") or 0.0
                                        :if price <= 0
                {return {"entry": None, "stop": None, "tp1": None, "tp2": None
        {return {"entry": price, "stop": price * 0.82, "tp1": price * 1.25, "tp2": price * 1.60

                                :def fmt_usd(value: Any) -> str
                                (value = finite_number(value
                                        :if value == 0
                                        "return "$0
                                :if abs(value) < 0.000001
                                "{return f"${value:.10g
                                        :if abs(value) < 0.01
                                "{return f"${value:.8f
                                        :if abs(value) < 1000
                        (".")return f"${value:,.6f}".rstrip("0").rstrip
                                "{return f"${value:,.0f

:def format_signal(candidate: dict[str, Any], assessment: dict[str, Any]) -> str
                                (levels = signal_levels(candidate
                                ("action = assessment.get("action", "WATCH
"مراقبة 👁️" if action == "SIGNAL" else "إشارة محاكاة تداول 🎯" = title
                                [] risks = assessment.get("risks") or
"لا توجد ملاحظات حرجية" join(str(x) for x in risks[:5]) or." ؛" = risk_text
```

```
    ) return
        "f"{title} — {candidate['name']} (${candidate['symbol']})\n
        f"العنوان: {candidate['mint']}\n"
        f"الإجراء: {action} | الثقة: {assessment.get('confidence', 0)}/100 | الدرجة:
{candidate['raw_score']}/100\n"
        f"السعر: {fmt_usd(candidate['price_usd'])} | العمر: {candidate.get('age_hours') or 0:.1f}
ساعة\n"
        f"السيولة: {fmt_usd(candidate['liquidity_usd'])} | حجم 1س:
{fmt_usd(candidate['volume_h1_usd'])}\n"
        f"التغير: 5د {candidate['change_m5_pct']:.2f}% | 1س {candidate['change_h1_pct']:.2f}%\n"
        f"التدفقات 1س: شراء {candidate['buys_h1']} / بيع {candidate['sells_h1']}\n"
        f"LP مقفلة: {candidate['lp_locked_pct']:.2f}%\n\n"
        f"الخلاصة: {assessment.get('thesis', '')}\n"
        f"المخاطر: {risk_text}\n"
        f"مستويات المحاكاة: دخول {fmt_usd(levels['entry'])} | وقف {fmt_usd(levels['stop'])} | TP1
{fmt_usd(levels['tp1'])} | TP2 {fmt_usd(levels['tp2'])}\n"
        f"الرابط: {candidate.get('pair_url') or ''}\n\n"
        "💡 محاكاة تداول افتراضية حية (Paper Trading). لا يتم تنفيذ صفقات حقيقية على المحفظة."
    )

# ---------------------------------------------------------- #
# Real-Time Paper Trading Simulation Engine #
# ---------------------------------------------------------- #
def paper_default_state() -> dict[str, Any]:
    initial = env_float("PAPER_INITIAL_BALANCE", 200.0)
    return {
        "mode": "paper",
        "enabled": True,
        "initial_balance": initial,
        "cash": initial,
        "realized_pnl": 0.0,
        "fees_paid": 0.0,
        "trades": [],
        "positions": {},
        "wins": 0,
        "losses": 0,
        "settings": {},
        "last_update": None,
    }

def load_paper_state() -> dict[str, Any]:
    default = paper_default_state()
    if not PAPER_STATE_FILE.exists():
        return default
    try:
        loaded = json.loads(PAPER_STATE_FILE.read_text(encoding="utf-8"))
        if not isinstance(loaded, dict) or loaded.get("mode") != "paper":
            return default
```

```
                                                    :()for key, value in default.items
                                                    (loaded.setdefault(key, value
                                                                return loaded
                                                              :except Exception
                                                                return default

                                                :def save_paper_state(state: dict[str, Any]) -> None
    PAPER_STATE_FILE.write_text(json.dumps(state, ensure_ascii=False, indent=2),
                                                                ("encoding="utf-8

                                                    :[def paper_cfg() -> dict[str, Any
                                                        ()state = load_paper_state
                                                    {} settings = state.get("settings") or
                                                                    } = cfg
                                            ,(risk_per_trade": env_float("PAPER_RISK_PER_TRADE", 0.03"
                                          ,(max_position_pct": env_float("PAPER_MAX_POSITION_PCT", 0.25"
                                              ,(max_positions": env_int("PAPER_MAX_POSITIONS", 4"
                                                  ,(fee_bps": env_float("PAPER_FEE_BPS", 30"
                                        ,(entry_slippage_bps": env_float("PAPER_ENTRY_SLIPPAGE_BPS", 50"
                                         ,(exit_slippage_bps": env_float("PAPER_EXIT_SLIPPAGE_BPS", 100"
                                             ,(max_hold_hours": env_float("PAPER_MAX_HOLD_HOURS", 12"
                                    ,(min_liquidity_factor": env_float("PAPER_MIN_LIQUIDITY_FACTOR", 0.5"
                                                                    {
                                                            (cfg.update(settings
                                                                return cfg

                    :def paper_equity(state: dict[str, Any], prices: dict[str, float] | None = None) -> float
                                                                {} prices = prices or
                                                        (("equity = finite_number(state.get("cash
                                                    :()for mint, pos in (state.get("positions") or {}).items
                                                price = prices.get(mint, finite_number(pos.get("last_price"),
                                                        ((((("finite_number(pos.get("entry_price
                                                    equity += finite_number(pos.get("quantity")) * price
                                                                return equity

                                                    :def paper_ledger(event: dict[str, Any]) -> None
                                                (PAPER_LEDGER_FILE.parent.mkdir(parents=True, exist_ok=True
                                                :with PAPER_LEDGER_FILE.open("a", encoding="utf-8") as handle
                                                    ("handle.write(json.dumps(event, ensure_ascii=False) + "\n

def paper_close(state: dict[str, Any], mint: str, price: float, fraction: float, reason: str, cfg:
                                                    :dict[str, Any], now: str) -> dict[str, Any] | None
                                                    (pos = (state.get("positions") or {}).get(mint
                                                                :if not pos or price <= 0
                                                                    return None
                                                            ((fraction = max(0.0, min(1.0, fraction
                                                quantity = finite_number(pos.get("quantity")) * fraction
                                                                :if quantity <= 0
```

```
                return None
            (exec_price = price * (1.0 - cfg["exit_slippage_bps"] / 10000.0
                gross = quantity * exec_price
                fee = gross * cfg["fee_bps"] / 10000.0
            cost = quantity * finite_number(pos.get("cost_per_unit"),
                (((("finite_number(pos.get("entry_price
                pnl = gross - fee - cost
            state["cash"] = finite_number(state.get("cash")) + gross - fee
        state["realized_pnl"] = finite_number(state.get("realized_pnl")) + pnl
            state["fees_paid"] = finite_number(state.get("fees_paid")) + fee
        (pos["quantity"] = max(0.0, finite_number(pos.get("quantity")) - quantity
                pos["last_price"] = price
    event = {"type": "close", "mint": mint, "symbol": pos.get("symbol"), "reason": reason,
"price": price, "execution_price": exec_price, "quantity": quantity, "pnl": pnl, "pnl_pct": (100.0 *
            {pnl / cost) if cost else 0.0, "fee": fee, "time": now
                (state.setdefault("trades", []).append(event
                    (paper_ledger(event
                :if pos["quantity"] <= 1e-12
                    (state["positions"].pop(mint, None
                        :if pnl >= 0
                state["wins"] = int(state.get("wins", 0)) + 1
                        :else
                state["losses"] = int(state.get("losses", 0)) + 1
                    return event

def paper_open(state: dict[str, Any], candidate: dict[str, Any], cfg: dict[str, Any], now: str) ->
                        :dict[str, Any] | None
                ("" mint = str(candidate.get("mint") or
                ((("price = finite_number(candidate.get("price_usd
            :({} ,"if not mint or price <= 0 or mint in state.get("positions
                    return None
            :["if len(state.get("positions") or {}) >= cfg["max_positions
                    return None
                (equity = paper_equity(state
                stop = price * 0.82
                ["risk_capital = equity * cfg["risk_per_trade
            (risk_per_unit = max(price - stop, price * 0.01
            quantity_by_risk = risk_capital / risk_per_unit
            ["max_notional = equity * cfg["max_position_pct
(notional = min(finite_number(state.get("cash")), max_notional, quantity_by_risk * price
                :if notional <= 1.0
                    return None
            (exec_price = price * (1.0 + cfg["entry_slippage_bps"] / 10000.0
                quantity = notional / exec_price
                fee = notional * cfg["fee_bps"] / 10000.0
                total = notional + fee
            :(("if total > finite_number(state.get("cash
                    return None
```

```
                state["cash"] = finite_number(state.get("cash")) - total
                state["fees_paid"] = finite_number(state.get("fees_paid")) + fee
                cost_per_unit = total / quantity
                } = pos
                    ,mint": mint"
                ,("symbol": candidate.get("symbol"
                    ,("name": candidate.get("name"
                    ,quantity": quantity"
                    ,entry_price": price"
                ,execution_entry_price": exec_price"
                    ,cost_per_unit": cost_per_unit"
                    ,notional": notional"
                ,stop_price": price * 0.82"
                ,tp1_price": price * 1.25"
                ,tp2_price": price * 1.60"
                    ,tp1_done": False"
                    ,opened_at": now"
                    ,last_price": price"
                ,highest_price": price"
                                {
            state.setdefault("positions", {})[mint] = pos
    event = {"type": "open", "mint": mint, "symbol": candidate.get("symbol"), "price": price,
    "execution_price": exec_price, "quantity": quantity, "notional": notional, "fee": fee, "time":
                                                                {now
            (state.setdefault("trades", []).append(event
                (paper_ledger(event
                    return event

def paper_process_results(results: list[dict[str, Any]], signal_threshold: float, min_liquidity:
                                                    :[[float) -> list[dict[str, Any
                    ()state = load_paper_state
                        ()cfg = paper_cfg
                    :(if not state.get("enabled", True
                                    [] return
                (now_dt = datetime.now(timezone.utc
                        ()now = now_dt.isoformat
    {("by_mint = {str(item.get("mint")): item for item in results if item.get("mint
                                    [] = [[events: list[dict[str, Any
                        ()closed_this_cycle: set[str] = set
                :(()for mint, pos in list((state.get("positions") or {}).items
                            (candidate = by_mint.get(mint
                price = finite_number((candidate or {}).get("price_usd"),
                                    (((("finite_number(pos.get("last_price
            (liquidity = finite_number((candidate or {}).get("liquidity_usd"), 0.0
                                    pos["last_price"] = price
        (pos["highest_price"] = max(finite_number(pos.get("highest_price")), price
    (("opened = datetime.fromisoformat(str(pos.get("opened_at")).replace("Z", "+00:00
                    (age = max(0.0, (now_dt - opened).total_seconds() / 3600.0
```

```
:(("if price <= finite_number(pos.get("stop_price
            (event = paper_close(state, mint, price, 1.0, "STOP_LOSS", cfg, now
                                                                    :if event
                                                        (events.append(event
                                                    (closed_this_cycle.add(mint
:(("elif price >= finite_number(pos.get("tp2_price
            (event = paper_close(state, mint, price, 1.0, "TAKE_PROFIT_2", cfg, now
                                                                    :if event
                                                        (events.append(event
                                                    (closed_this_cycle.add(mint
:("elif price >= finite_number(pos.get("tp1_price")) and not pos.get("tp1_done
            (event = paper_close(state, mint, price, 0.5, "TAKE_PROFIT_1", cfg, now
                                                                    :if event
                                                        (events.append(event
                                                (pos = state.get("positions", {}).get(mint
                                                                    :if pos
                                                            pos["tp1_done"] = True
                                pos["stop_price"] = max(finite_number(pos.get("stop_price")),
                                                        (((("finite_number(pos.get("entry_price
                                                                :["elif age >= cfg["max_hold_hours
            (event = paper_close(state, mint, price, 1.0, "TIME_EXIT", cfg, now
                                                                    :if event
                                                        (events.append(event
                                                    (closed_this_cycle.add(mint
:["elif candidate and liquidity > 0 and liquidity < min_liquidity * cfg["min_liquidity_factor
            (event = paper_close(state, mint, price, 1.0, "LIQUIDITY_EXIT", cfg, now
                                                                    :if event
                                                        (events.append(event
                                                    (closed_this_cycle.add(mint
                                                                :for candidate in results
                                                {} assessment = candidate.get("assessment") or
if candidate.get("mint") not in closed_this_cycle and candidate.get("hard_gate_passed")
  and assessment.get("action") == "SIGNAL" and finite_number(candidate.get("raw_score"))
                                                                :>= signal_threshold
                                            (event = paper_open(state, candidate, cfg, now
                                                                    :if event
                                                        (events.append(event
                                                            state["last_update"] = now
                                                [:state["trades"] = state.get("trades", [])[-2000
                                                        (save_paper_state(state
                                                            return events

                                                :def pnl_marker(value: float) -> str
                                                                :if value > 1e-12
                                                            "ربح 🟩" return
                                                                :if value < -1e-12
                                                        "خسارة 🟥" return
                                                            "تعادل ⬜" return
```

```
:def format_paper_event(event: dict[str, Any]) -> str
(((("?" ,"symbol = html.escape(str(event.get("symbol
:"if event.get("type") == "open
) return
"b>\n/>📈<b>" محاكاة — فتح صفقة جديدة
"b>{fmt_usd(event.get('notional'))}</b>\n> القيمة: <f"<b>${symbol}</b
f"سعر السوق: {(('fmt_usd(event.get('price}} | التنفيذ:
"fmt_usd(event.get('execution_price'))}\n}
f"الرسوم: {\n}"fmt_usd(event.get('fee'))
"صفقة افتراضية آمنة."
(
(("pnl = finite_number(event.get("pnl
(marker = pnl_marker(pnl
) return
"b>\n/>📉f"<b محاكاة — إغلاق صفقة
code>{html.escape(str(event.get('reason',> السبب: <f"<b>${symbol}</b
"'UNKNOWN'))))}</code>\n
f"سعر السوق: {\n}"fmt_usd(event.get('price'))
"f"{marker}: <b>{fmt_usd(pnl)}</b> ({finite_number(event.get('pnl_pct')):+.2f}%)\n
f"الرسوم: {(('fmt_usd(event.get('fee}}"
(

:def paper_status() -> str
()state = load_paper_state
()cfg = paper_cfg
prices = {mint: finite_number(pos.get("last_price")) for mint, pos in (state.get("positions")
{()or {}).items
(equity = paper_equity(state, prices
(initial = finite_number(state.get("initial_balance"), 200.0
pnl = equity - initial
["trades = [x for x in state.get("trades", []) if x.get("type") == "close
winrate = (100.0 * int(state.get("wins", 0)) / len(trades)) if trades else 0.0
] = lines
"<b>📊 تقرير محاكي التداول الورقي (Paper Trading)</b>",
f"الحالة: {'مفعّلة' if state.get('enabled', True) else 'متوقفة'}",
f"الرصيد الابتدائي: {(fmt_usd(initial}} | القيمة الحالية: {(fmt_usd(equity}}",
,"<f"{pnl_marker(pnl)}: <b>{fmt_usd(pnl)}</b
f"النقد المتاح: {(('fmt_usd(state.get('cash}} | الربح المحقق: {(('fmt_usd(state.get('realized_pnl}}",
f"الرسوم: {(('fmt_usd(state.get('fees_paid}} | صفقات مغلقة: {(len(trades}} | نسبة النجاح:
,"%{winrate:.1f}
[
:({} if not (state.get("positions") or
lines.append("\nلا توجد صفقات مفتوحة حالياً.")
:()for pos in (state.get("positions") or {}).values
(("price = finite_number(pos.get("last_price
(("quantity = finite_number(pos.get("quantity
(mark_price = price * (1.0 - cfg["exit_slippage_bps"] / 10000.0
```

```
                (net_value = quantity * mark_price * (1.0 - cfg["fee_bps"] / 10000.0
                    cost_basis = quantity * finite_number(pos.get("cost_per_unit"),
                                        (((("finite_number(pos.get("entry_price
                                                open_pnl = net_value - cost_basis
                                    open_pct = 100.0 * open_pnl / cost_basis if cost_basis else 0.0
                                                )lines.append
                        "html.escape(str(pos.get('symbol', '?'))))}</b>\n}$ مركز مفتوح"f"\n<b
                                                "fmt_usd(net_value)}\n}القيمة المحاكية: "f
                "f"{pnl_marker(open_pnl)}: <b>{fmt_usd(open_pnl)}</b> ({open_pct:+.2f}%)\n
                "fmt_usd(pos.get('last_price'))}\n}الحالي: {(('fmt_usd(pos.get('entry_price}الدخول: "f
                fmt_usd(pos.get('stop_price'))} | TP1: {fmt_usd(pos.get('tp1_price'))} | TP2:}وقف: "f
                                                "{(('{fmt_usd(pos.get('tp2_price
                                                        (
                                                (return "\n".join(lines


                                                ---------- #
                                                Telegram Bot Core & Control #
                                                ---------- #
def send_telegram(text: str, cfg: dict[str, Any], *, parse_mode: str | None = None) -> dict[str,
                                                        :[Any
                                                ("token = os.getenv("TELEGRAM_BOT_TOKEN
                                        (chat_id = os.getenv("TELEGRAM_CHAT_ID", DEFAULT_CHAT_ID
                                                        :if not token
                                                ("غير محدد raise RuntimeError("TELEGRAM_BOT_TOKEN
                                        {[payload: dict[str, Any] = {"chat_id": chat_id, "text": text[:4090
                                                        :if parse_mode
                                                payload["parse_mode"] = parse_mode
(return post_json(f"{TELEGRAM_BASE}/bot{token}/sendMessage", payload, timeout=30


                                                :[def load_state() -> dict[str, Any
                                                :()if not STATE_FILE.exists
    return {"sent_mints": {}, "last_scan": None, "enabled": True, "alerts_enabled": True,
                                                        {{} :""settings
                                                                :try
                                                (("data = json.loads(STATE_FILE.read_text(encoding="utf-8
return data if isinstance(data, dict) else {"sent_mints": {}, "last_scan": None, "enabled":
                                                        {{} :"True, "alerts_enabled": True, "settings
                                                                :except Exception
    return {"sent_mints": {}, "last_scan": None, "enabled": True, "alerts_enabled": True,
                                                        {{} :""settings

                                                :def save_state(state: dict[str, Any]) -> None
                                        STATE_FILE.write_text(json.dumps(state, ensure_ascii=False, indent=2),
                                                        ("encoding="utf-8

                                                :[def runtime_cfg() -> dict[str, Any
                                                        ()state = load_state
                                                {} settings = state.get("settings") or
```

```
} = cfg
,(min_liquidity": env_float("MIN_LIQUIDITY_USD", 5000"
,(min_h1_volume": env_float("MIN_H1_VOLUME_USD", 5000"
,(max_age_hours": env_float("MAX_SIGNAL_AGE_HOURS", 48"
,(max_top_holder_pct": env_float("MAX_TOP_HOLDER_PCT", 40"
,(max_candidates": env_int("MAX_CANDIDATES", 8"
,(signal_threshold": env_float("SIGNAL_THRESHOLD", 65"
,{"ai_enabled": os.getenv("AI_ENABLED", "false").lower() in {"1", "true", "yes"
{
(cfg.update(settings
return cfg

:[[def scan_once(*, send: bool = False, dry_run: bool = False) -> list[dict[str, Any
()cfg = runtime_cfg
()profiles = fetch_profiles
([pairs_by_token = fetch_pairs([p["tokenAddress"] for p in profiles
[] = [[[ranked: list[tuple[float, dict[str, Any], dict[str, Any], dict[str, Any
:for profile in profiles
((([] ,["pair = choose_pair(pairs_by_token.get(profile["tokenAddress
:if not pair
continue
rough = nested_number(pair.get("liquidity"), "usd") + nested_number(pair.get("volume"),
(""h1
(({} ,ranked.append((rough, profile, pair
(ranked.sort(key=lambda x: x[0], reverse=True
[] = [[results: list[dict[str, Any
:[["for _, profile, pair, _ in ranked[: cfg["max_candidates
:try
(["report = rug_report(profile["tokenAddress
(candidate = normalize_candidate(profile, pair, report, cfg
} assessment = ai_assess(candidate, cfg) if candidate["hard_gate_passed"] else
,"action": "AVOID"
,confidence": 100"
"thesis": "تم الرفض بواسطة بوابة المخاطر.",
"risks": candidate.get("gate_reasons"), [] ,
"invalidation": "N/A", "entry_plan": "لا دخول", "exit_plan": "لا صفقة", "rationale": "خطر
حتمي"
{
candidate["assessment"] = assessment
(results.append(candidate
:except Exception as exc
(LOG.warning("candidate process failed: %s", exc
(REPORTS.mkdir(parents=True, exist_ok=True
report_path = REPORTS /
"f"scan_{datetime.now(timezone.utc).strftime('%Y%m%dT%H%M%SZ')}.json
report_path.write_text(json.dumps(results, ensure_ascii=False, indent=2),
("encoding="utf-8
()state = load_state
```

```
                ()state["last_scan"] = datetime.now(timezone.utc).isoformat
                                ({} ,"state.setdefault("sent_mints
                                        :for candidate in results
                                {} assessment = candidate.get("assessment") or
                                                        ) = should_send
                                                send and not dry_run
                                        ("and candidate.get("hard_gate_passed
                                    "and assessment.get("action") == "SIGNAL
                            ["and candidate.get("raw_score", 0) >= cfg["signal_threshold
                                    ["and candidate.get("mint") not in state["sent_mints
                                                                        (
                                                        :if should_send
                                    (send_telegram(format_signal(candidate, assessment), cfg
                        ()state["sent_mints"][candidate["mint"]] = datetime.now(timezone.utc).isoformat
                                                        :if not dry_run
                                                    (save_state(state
                                                            :try
                        paper_events = paper_process_results(results, cfg["signal_threshold"],
                                                                (["cfg["min_liquidity
                                                    :for event in paper_events
                            ("send_telegram(format_paper_event(event), cfg, parse_mode="HTML
                                                    :except Exception as exc
                                                (LOG.exception("paper engine failed: %s", exc
                                                        return results

:[[def guarded_scan_once(*, send: bool = False, dry_run: bool = False) -> list[dict[str, Any
                                                    :(if not SCAN_LOCK.acquire(blocking=False
                                                    ("فحص جارٍ بالفعل")raise RuntimeError
                                                            :try
                                                (return scan_once(send=send, dry_run=dry_run
                                                            :finally
                                                        ()SCAN_LOCK.release

                                    :def authorized_update(update: dict[str, Any]) -> bool
                                            {} message = update.get("message") or
                                    (("" ,"chat_id = str((message.get("chat") or {}).get("id
                                    (("" ,"user_id = str((message.get("from") or {}).get("id
                            ((allowed = str(os.getenv("TELEGRAM_CHAT_ID", DEFAULT_CHAT_ID
                                (return bool(allowed) and (chat_id == allowed or user_id == allowed

                                    :def handle_command(update: dict[str, Any]) -> None
                                                :(if not authorized_update(update
                                                        return
                                            {} message = update.get("message") or
                                        ()text = str(message.get("text") or "").strip
                                                :("/")if not text.startswith
                                                        return
                                            (" ")command, _, args = text.partition
```

```
                ()command = command.split("@", 1)[0].lower
chat_id = str((message.get("chat") or {}).get("id") or os.getenv("TELEGRAM_CHAT_ID",
                                                    ((DEFAULT_CHAT_ID
                                                :{"if command in {"/start", "/help
                                                        ) = reply
                                                "🤖 **أوامر التحكم بالبوت والمحاكي:**\n\n"
                                                "/paper_status — عرض حالة وتقارير المحاكي الحية\n"
                                                "/scan — تشغيل فحص فوري وتقييم الفرص\n"
                                                "/status — عرض إعدادات وحالة البوت\n"
                                                "/pause أو /resume — إيقاف/استئناف الفحص\n"
                                                "/paper_reset — إعادة تصفير رصيد المحاكاة إلى 200 USDC"
                                                                                        )
                                                    :"elif command == "/paper_status
                                                            ()reply = paper_status
                                                        :"elif command == "/status
                                                                ()state = load_state
                                                                ()cfg = runtime_cfg
                                                    :reply = f"حالة البوت: {'مفعّل' if state.get('enabled') else 'متوقف'}\nآخر فحص
{state.get('last_scan')}\nالسيولة الأدنى: ${cfg['min_liquidity']}"
                                                        :"elif command == "/paper_reset
                                                        (()save_paper_state(paper_default_state
                                                                :()if PAPER_LEDGER_FILE.exists
                                                                ()PAPER_LEDGER_FILE.unlink
                                                    reply = "تمت إعادة ضبط حساب المحاكاة إلى 200 USDC افتراضية."
                                                            :"elif command == "/scan
                                                    send_telegram("⏳ جارٍ الفحص وتحديث المحاكي...", {}, parse_mode=None)
                                                                        :try
                                                                (guarded_scan_once(send=False
                                                                    ()reply = paper_status
                                                                    :except Exception as e
                                                                "{e}خطأ أثناء الفحص: "reply = f
                                                                        :else
                                                            reply = "أمر غير معروف. استخدم /help"
final_payload = {"chat_id": chat_id, "text": reply, "parse_mode": "HTML" if command ==
                                                    {"/paper_status" else None

post_json(f"{TELEGRAM_BASE}/bot{os.getenv('TELEGRAM_BOT_TOKEN')}/sendMessage
                                                    (", final_payload

                                                :def telegram_poll_loop() -> None
                                                            offset = 0
                                                ("token = os.getenv("TELEGRAM_BOT_TOKEN
                                                :()while not STOP_EVENT.is_set
                                                                    :try
                                                            data =
("get_json(f"{TELEGRAM_BASE}/bot{token}/getUpdates?offset={offset}&timeout=20
                                                :([] ,"for update in (data or {}).get("result
                                                (offset = max(offset, int(update.get("update_id", 0)) + 1
```

```
()threading.Thread(target=handle_command, args=(update,), daemon=True).start
                                                                    :except Exception
                                                                      (time.sleep(5

                                                                        :def main() -> int
                                                                         ()load_dotenv
                    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
                                                                          ("%(message)s
                                                                       ()start_health_server
                                ()threading.Thread(target=telegram_poll_loop, daemon=True).start
                                        LOG.info("Bot started successfully. Targeting Chat ID: %s",
                                          ((os.getenv("TELEGRAM_CHAT_ID", DEFAULT_CHAT_ID
                                        ((poll_seconds = max(60, env_int("POLL_SECONDS", 120
                                                                    :()while not STOP_EVENT.is_set
                                                                                    :try
                                                                          ()state = load_state
                                                                   :(if state.get("enabled", True
                                                ((guarded_scan_once(send=state.get("alerts_enabled", True
                                                                    :except Exception as exc
                                                            (LOG.exception("Scan loop failed: %s", exc
                                                                    (time.sleep(poll_seconds
                                                                                  return 0

                                                                     :"__if __name__ == "__main
                                                                          (()sys.exit(main
```